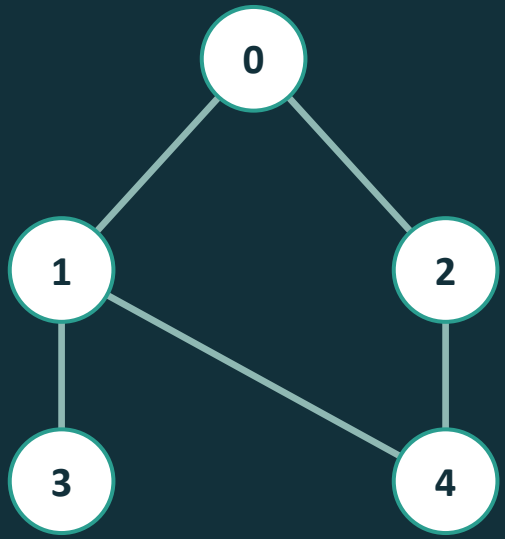
Depth First Search

Graph traversal in Design and Analysis of Algorithms, implemented in C

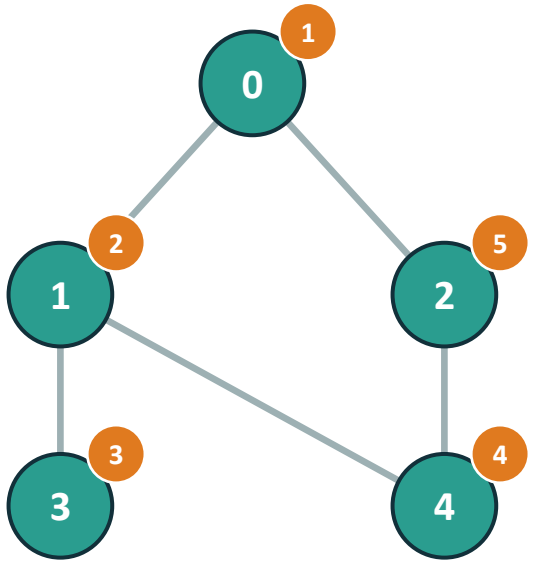


DAA | Unit: Graph Algorithms

What is DFS?

- Go as deep as possible along one path before backing up
- When a vertex has no unvisited neighbour, backtrack
- Uses a stack: the recursion call stack, or an explicit one
- A visited[] array stops us from looping forever

Think of exploring a maze: keep walking until a dead end, then return to the last junction.



Orange badges show visit order from vertex 0

The algorithm

1. Mark the current vertex visited
2. Process it (print or record)
3. Recurse on each unvisited neighbour
4. Backtrack when none are left

```
DFS(G, u)
  visited[u] = true
  for each v in Adj[u]
    if not visited[v]
      DFS(G, v)
```

For a disconnected graph, call DFS from every unvisited vertex.

DFS in C (adjacency matrix)

```c
#include <stdio.h>
#define MAX 10

int adj[MAX][MAX], visited[MAX], n;

void dfs(int u) {
    visited[u] = 1;
    printf("%d ", u);
    for (int v = 0; v < n; v++)
        if (adj[u][v] && !visited[v])
            dfs(v);
}

int main() {
    int start;
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    scanf("%d", &start);
    dfs(start);
    return 0;
}
```

visited[]

Global array, all 0 at start

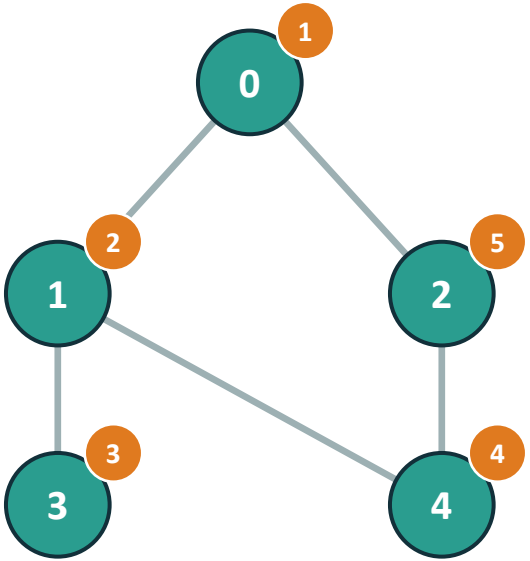
Recursion

The call stack is our stack

adj[u][v]

1 means an edge exists

Dry run from vertex 0



Call	Action	Output
dfs(0)	Mark 0, go to 1	0
dfs(1)	Mark 1, go to 3	0 1
dfs(3)	No new neighbour, backtrack	0 1 3
dfs(4)	Back in 1, next is 4	0 1 3 4
dfs(2)	From 4, go to 2	0 1 3 4 2
return	All calls unwind	Done

Edges: 0-1, 0-2, 1-3, 1-4, 2-4. Neighbours visited in ascending order.

Complexity

$O(V^2)$

Time with adjacency matrix

Scan a full row of V entries for each of V vertices

$O(V + E)$

Time with adjacency list

Each vertex and each edge is handled once

$O(V)$

Space

visited[] plus recursion depth, up to V in the worst case

Where DFS is used

Cycle detection

Spot back edges in a graph

Topological sort

Order tasks in a DAG

Components

Group reachable vertices

Path finding

Solve mazes and puzzles

Strong components

Kosaraju and Tarjan

Bipartite check

Colour while traversing

DFS goes deep with a stack; BFS goes level by level with a queue.